

Architecture Specification Document

Project: Programmable Development Environment

Version: 1.0

1. Vision

Purpose

Build a next-generation programmable development environment combining:

- the extensibility philosophy of Emacs
- the efficiency and modal interaction of Vim
- the language intelligence of modern IDEs
- the performance and safety of Rust
- the behavioral programmability of Janet

The system should not be designed as a text editor with plugins.

The system should be designed as a **development environment kernel** where text, code, projects, runtime state, and tools are first-class programmable objects.

2. Core Principles

2.1 Small Native Core

The Rust core provides:

- performance-critical operations
- memory safety
- concurrency
- rendering
- storage
- core data structures

The core should remain stable and minimal.

2.2 Everything Is An Object

The system models:

- files
- buffers
- symbols
- projects
- processes
- debugger states
- commands
- views

as objects accessible through APIs.

Example:

```
Buffer
|
+ text
+ language
+ symbols
+ diagnostics
+ history
```

2.3 Event Driven Architecture

All subsystems communicate through events.

Example:

```
buffer.changed

symbol.indexed

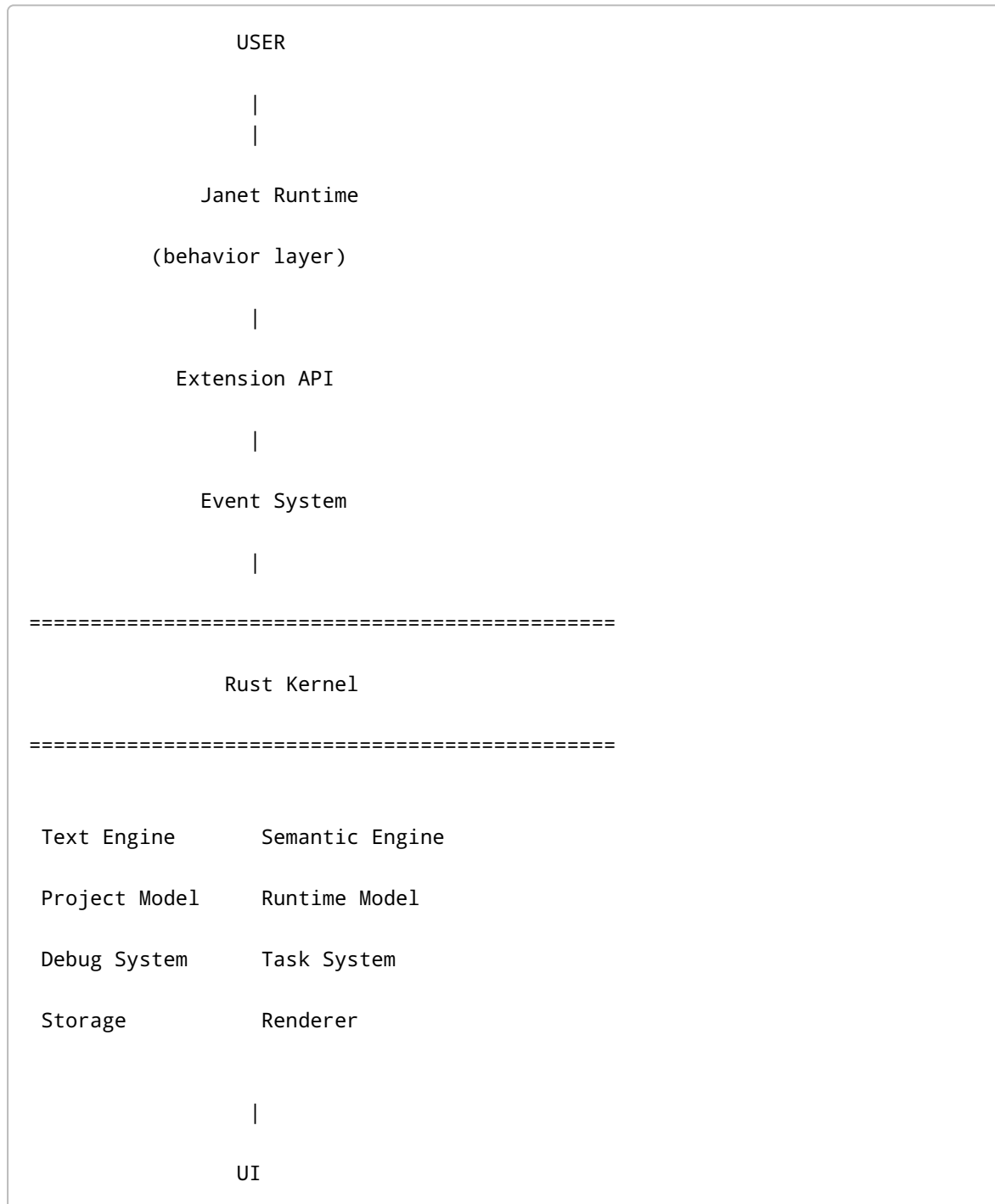
debug.paused

test.completed

project.updated
```

Components react independently.

3. High Level Architecture



4. System Components

4.1 Rust Kernel

The kernel is responsible for:

- lifecycle management
- memory ownership
- threading
- communication
- persistence

Directory:

```
kernel/  
  
    event/  
    runtime/  
    scheduler/  
    storage/  
    ipc/  
    config/
```

4.2 Text Engine

The text engine manages all textual content.

Responsibilities:

- buffers
- editing
- undo/redo
- selections
- cursors
- transformations

Architecture:

```
Text Engine
```

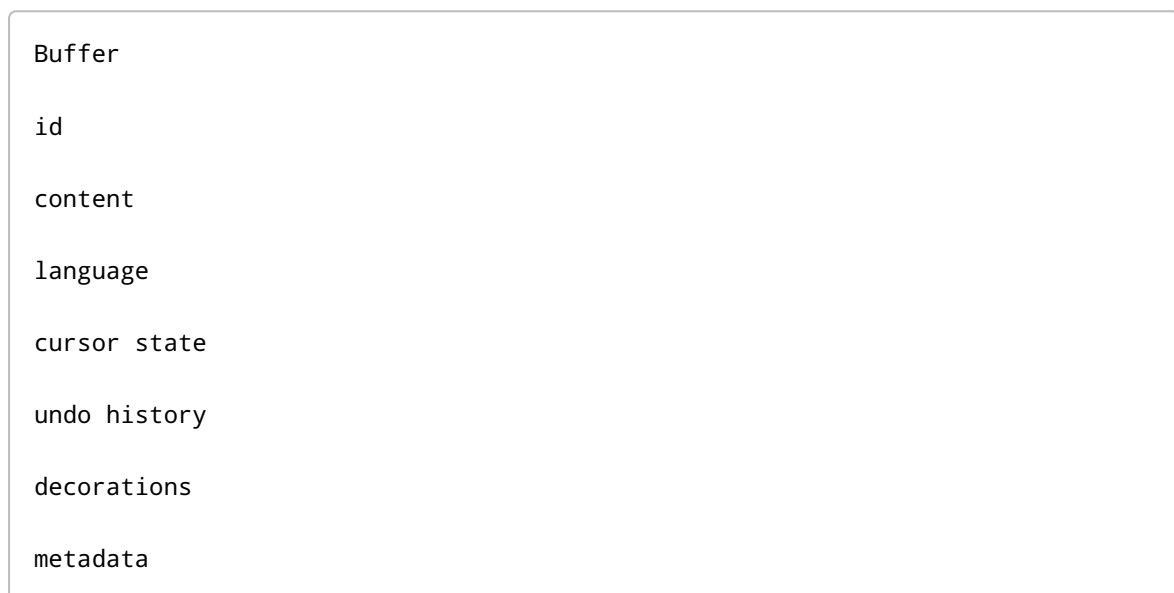
```
    Buffer
```

```
        |  
        |
```



Buffer Model

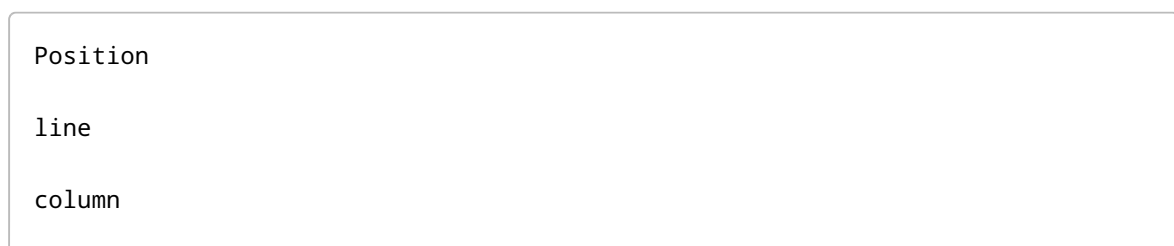
Example:



Position Model

Never use raw indexes.

Use:



offset

5. Rendering Architecture

The renderer is independent from text.

Architecture:

```
Model
|
View Tree
|
Renderer
|
Terminal / GUI
```

View Objects

Examples:

```
EditorView
SidebarView
TerminalView
DebuggerView
ProjectView
```

Each view consumes models.

6. Gutter System

The editor must support programmable gutters.

Architecture:

Gutter

|
+ Line numbers

|
+ Git markers

|
+ Diagnostics

|
+ Breakpoints

|
+ Coverage

Each gutter provider implements:

render()

width()

update()

7. Semantic Engine

This is the intelligence layer.

Purpose:

Convert text into knowledge.

Pipeline:

Source Code

|

Parser

|

Symbols

|

Relationships

|

Project Graph

Semantic Objects

Example:

Function

name

location

references

documentation

tests

history

Capabilities:

- navigation
- refactoring
- code intelligence
- AI context
- visualization

8. Language Architecture

Languages are external modules.

Structure:


```
languages/  
  
  rust/  
  
  python/  
  
  java/  
  
  javascript/
```

Each provides:

```
Language Provider  
  
  parser  
  
  formatter  
  
  diagnostics  
  
  completion  
  
  debugger adapter
```

Interface:

```
Language {  
  
  parse()  
  
  symbols()  
  
  diagnostics()  
  
  format()  
  
}
```

9. Project System

Projects are first-class objects.

Model:

Project

|

+ Modules

|

+ Dependencies

|

+ Tasks

|

+ Build Targets

|

+ Runtime Config

Example:

```
project.build()
```

```
project.test()
```

```
project.run()
```

10. Task System

Everything executable is a task.

Examples:

Build

Test

Deploy

Format

Generate

Model:

Task

name

command

environment

dependencies

output

11. Debug Architecture

Debugger is a core subsystem.

Architecture:

Debug Manager

|

Debug Session

|

Process

|

Threads

|

Frames

|

Variables

Objects:

Breakpoint

StackFrame

Variable

MemoryRegion

Thread

Features:

- breakpoints
- stepping
- watches
- evaluation
- timeline debugging

12. Plugin System

Plugins do not modify internals.

They use capabilities.

Architecture:

Plugin

|

+ Commands

+ Events

+ Views

+ Providers

+ Models

13. Janet Runtime

Janet is the behavioral layer.

Responsibilities:

- customization
- automation
- workflows
- extensions

Examples:

```
commands  
  
keybindings  
  
hooks  
  
automation  
  
UI behavior
```

Example:

```
on file.save:  
  
  format  
  
  run tests  
  
  update docs
```

14. Command System

Everything should be a command.

Examples:

```
open-file  
  
rename-symbol  
  
run-tests  
  
debug-start
```

```
project-build
```

Commands are discoverable.

15. Workspace Model

The workspace stores user state.

Includes:

```
open buffers  
  
layout  
  
history  
  
projects  
  
settings  
  
semantic cache
```

Example:

```
workspace.save()  
  
workspace.restore()
```

16. AI / Agent Architecture

AI is integrated as a system participant.

Not a chat window.

Architecture:

```
Agent  
  
|  
  
Context Provider
```

```
|
Planner
|
Executor
```

Agents use editor APIs:

```
read_symbol()

modify_buffer()

run_task()

inspect_debug()
```

17. Storage Architecture

Persistent storage:

```
workspace/
  buffers/
  history/
  index/
  settings/
  sessions/
```

18. Concurrency Model

Use asynchronous execution.

Example:

```
Main Thread
```

UI

Worker Threads

indexing

parsing

builds

diagnostics

19. Security Model

Extensions run with controlled permissions.

Capabilities:

filesystem

network

process execution

editor access

20. Development Roadmap

Phase 1

Core:

- text engine
- renderer
- events
- Janet embedding

Phase 2

Developer features:

- LSP
- project model
- tasks

- debugger

Phase 3

Semantic environment:

- project graph
- advanced navigation
- refactoring

Phase 4

Programmable ecosystem:

- packages
- agents
- workflows

Final Architecture Philosophy

The system is built around one idea:

The editor is not a place where users edit files.

It is a programmable environment where users create their own development system.

Rust provides the foundation.

Janet provides the behavior.

Everything else is a model built on top.